中国大学生计算机设计大赛

软件开发类作品文档简要要求

作品编号：2026011230

作品名称：智销预策

作　　者：方阳 丁鹏飞 陈泽鹏

版本编号：1.0.0

填写日期：2026-04-0

填写说明：

1、本文档适用于所有涉及软件应用与开发类的各个小类作品，包括：（1）Web 应用与开发（2）管理信息系统（3）移动应用开发（非游戏类）（4）算法设计与应用（5）企业赛项；

2、本文档为简要文档，不宜长篇大论，需简明扼要，建议设计二级目录，逻辑性强；

3、一级标题采用二号黑体，居中，二级标题采用三号黑体，靠左，根据需要可以设计三级标题，正文一律用五号宋体；

4、提交文档时，以 PDF 格式提交本文档；

5、本文档内容是正式参赛内容组成部分，务必真实填写。如不属实，将导致奖项等级降低甚至终止本作品参加比赛

目　录

第一章　需求分析 3

第二章　概要设计 4
第三章　详细设计 5
第四章　测试报告 6
第五章　安装及使用 8
第六章　项目总结 9
参考文献..... 10

第一章 需求分析

1.1 开发背景与目的

针对中小企业在销售管理中存在的“数据孤岛”与“库存积压”痛点，本作品旨在开发一套轻量化、智能化的 SaaS 销售数据平台。对标传统昂贵的企业级 ERP，本平台侧重通过**AI 算法（LightGBM）**实现销量预测，帮助用户从被动记录转向主动决策，降低运营成本。

1.2 用户与功能

面向用户：中小零售商、电商运营人员及仓储管理者。

主要功能：支持多租户数据隔离；涵盖商品、销售、库存全流程管理；集成 AI 销量预测及多维度数据可视化大屏。

1.3 主要性能

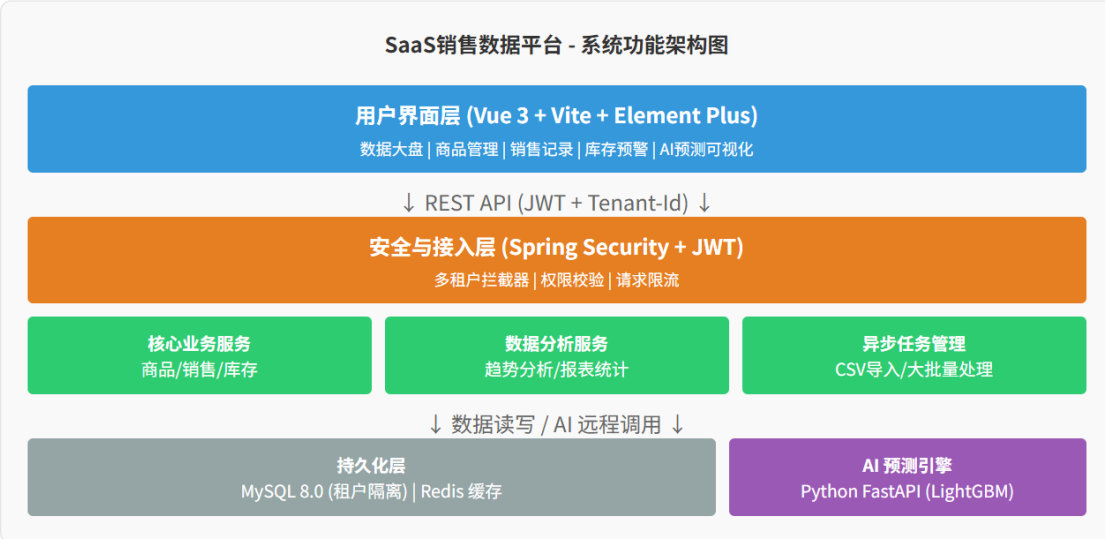
系统采用 Spring Boot 3 + FastAPI 架构。使用 redis 作为缓存，核心接口响应小于 300ms；支持 JWT 无状态认证与多租户隔离，确保数据安全；大批量数据导入及 AI 预测采用异步处理，保障高并发下的稳定性。

1.4 竞品分析

维度	本作品（SaaS AI 平台）	传统中小型 ERP	Excel/手工记录
智能决策	集成 AI 模型，支持销量预测	无/需人工计算	无
部署成本	低（开通即用，SaaS 模式）	高（需服务器部署）	极低
数据安全	强（多租户逻辑隔离）	一般	易丢失/难审计

第二章 扩展性微服务架构，易于扩展 架

构陈旧，升级难 无概要设计



人机交互：前端 Vue 3 通过 Axios 异步调用后端 REST 接口，并在 Header 中携带 Authorization (Token) 和 X-Tenant-Id (租户标识)。

租户隔离：后端拦截器提取租户 ID，利用 MyBatis Plus 插件动态拼接 SQL 过滤条件。

AI 集成：当用户发起预测请求时，后端 Service 封装历史销售数据，通过 HTTP 请求发送至 Python FastAPI 服务，返回预测结果。

第三章 异步机制：大批量导入数据时，Controller 立即返回 JobId，由异步线程处理，前端通过轮询接口查询任务进度。详细设计

3.1 界面设计与业务流程

本平台采用 Vue 3 + Element Plus 构建，核心界面遵循“直观、扁平、数据驱动”原则。

核心布局：左侧导航栏 + 顶部租户信息 + 右侧数据大盘。

3.2 数据库设计

系统基于 MySQL 8.0，核心逻辑是利用 tenant_id 字段实现物理表内的逻辑隔离。

3.2.1 核心 E-R 关系

租户 (Tenant)：管理所有基础数据。

商品 (Product) 1 : N 销售数据 (Sales)。

商品 (Product) 1 : 1 库存 (Inventory)。

3.2.2 核心数据表（销售数据表 sales_data）

字段名　类型　说明

id　BIGINT (PK) 主键 ID

tenant_idBIGINT　租户标识（数据隔离核心）

product_id　BIGINT　关联商品 ID

quantity INT 销售数量

sale_dateDATE　销售日期（索引，优化查询速度）

3.3 关键技术与创新点

1. 透明化多租户隔离

实现：基于 MyBatis Plus 拦截器。

特点：在 SQL 执行前自动植入 tenant_id 过滤条件。开发者编写业务代码时无需关心租户逻辑，从底层杜绝了数据越权访问。

2. 轻量级 AI 销量预测 (LightGBM)

实现：Java 后端通过 REST API 调用 Python 预测服务。

创新：相比传统统计学模型，LightGBM 能处理更多非线性特征（如季节、价格波动），为中小企业提供更精准的补货参考。

3. 大规模数据异步导入

难点：万级以上销售数据导入会导致前端卡死或请求超时。

解决：采用"任务 ID + 异步线程池"模式。用户上传 CSV 后立即返回 job_id，系统后台静默处理并支持进度查询，保障了系统的高可用性。

4. 实时库存预警机制

技术：Redis 缓存 + 安全水位算法。

第四章 效果：当销售导致库存低于 safety_stock 时，系统自动触发预警提醒，减少人工盘点误差。测试报告

4.1 主要测试过程与结果

由于本平台为前后端分离架构，主要通过 Postman（接口测试）和 浏览器开发者工具（前端联调）进行黑盒测试，验证核心业务逻辑与多租户隔离。

4.1.1 核心测试用例

测试模块 测试场景 预期结果 实际结果

认证授权 无 Token 访问 /api/products 返回 401 Unauthorized 符合预期

多租户隔离 租户 A 查询商品列表 仅能看到属于租户 A 的商品 符合预期

AI 预测 输入 6 个月的历史销售记录 返回未来 30 天销量趋势数组 符合预期

异步导入 上传 5000 条销售记录 CSV 立即返回 JobId，后台 5s 内处理完成 符合预期

库存预警 销售后库存低于安全水位 系统自动标记该商品为"低库存" 符合预期

4.1.2 缺陷发现与修正

发现问题：在初始阶段，批量导入 10000 条数据时，由于 Spring Boot 默认数据库连接池配置过小，导致连接溢出报错。

修正措施：在 application.yml 中调优 HikariCP 连接池参数（增加 max-lifetime 和 maximum-pool-size），并将数据库写入操作改为批量插入 (Batch Insert)，导入速度提升了 3 倍。

4.2 多维度技术指标

根据实测与系统架构分析，得出以下技术指标：

1. 运行速度 (Performance)

API 响应：核心业务接口 (CRUD) 平均响应时间 < 200ms。

AI 预测时延：从发起请求到 Python 引擎返回结果，端到端耗时 < 1.5s。

高并发支持：引入 Redis 缓存高频访问的库存数据，显著降低数据库压力。

2. 安全性 (Security)

双重隔离：应用层通过 JWT 进行身份校验，持久层通过 MyBatis 插件实现租户级 SQL 逻辑隔离。

密码安全：用户密码采用 BCrypt 强哈希加密存储，不可逆向破解。

3. 部署方便性 (Deployment)

容器化支持：提供 Dockerfile，支持一键打包镜像。

环境依赖：仅需标准 JDK 17 与 MySQL 8.0 即可运行，不依赖复杂的中间件。

4. 扩展性 (Scalability)

解耦架构：Java 业务逻辑与 Python AI 预测服务解耦，可根据算力需求独立进行横向扩展。

异步能力：采用任务队列模式处理耗时操作，确保系统不因大数据量任务而阻塞。

5. 可用性 (Availability)

容错处理：全局异常拦截器确保系统在出错时返回统一的 JSON 错误信息，前端通过全局 Toast 提示，保障用户体验连贯。

第五章 安装及使用

5.1 环境要求

为保证系统稳定运行，建议在以下环境中部署：

操作系统：Windows 10/11、Ubuntu 20.04+ 或 macOS。

运行时环境：

Java: JDK 17+

Python: 3.9+ (用于 AI 预测服务)

Node.js: 16.0+ (用于前端构建)

数据库与中间件：

MySQL: 8.0+

Redis: 6.0+

5.2 安装过程

1. 数据库准备

创建数据库 sales_platform，字符集选择 utf8mb4。

执行项目根目录下的 db/init.sql 脚本，初始化表结构及基础数据。

2. 后端服务启动 (Java)

修改 src/main/resources/application.yml 中的 MySQL 和 Redis 连接地址及密码。

在项目根目录执行：mvn clean install。

运行主类 SalesPlatformApplication.java 或执行 java -jar target/sales-platform.jar。

3. AI 预测服务启动 (Python)

进入 AiPredict/ 目录。

安装依赖：pip install -r requirements.txt。

启动服务：uvicorn main:app --port 5000。

4. 前端页面启动 (Vue)

进入 frontend/ 目录。

安装依赖：npm install 或 pnpm install。

启动开发服务器：npm run dev。

访问地址：http://localhost:5173。

5.3 典型使用流程

本平台采用标准的 SaaS 业务逻辑，推荐使用流程如下：

账号注册与初始化：

访问登录页进行注册，系统将自动分配唯一的 Tenant ID（租户标识）。

登录系统后，进入“商品管理”录入企业基础商品信息。

数据录入与同步：

手动录入：在“销售管理”模块逐条添加每日销售流水。

批量导入：下载 CSV 模板，上传历史销售数据，系统通过异步任务自动更新库存。

智能分析与预测：

查看“数据大盘”，实时监控销售趋势及库存水位。

进入“AI 预测”模块，点击“生成预测”，查看基于 LightGBM 算法的未来销量建议。

库存预警响应：

当系统提示“低库存预警”时，根据 AI 建议的采购量进行补货。

第六章 在“库存管理”中手动更新到货数量，完成闭环管理。项目总结

6.1 开发历程与任务分解

本项目的开发经历了从需求调研、架构设计到前后端联调的完整生命周期。在任务分解上，我采用了**“核心先行，模块迭代”**的策略：

第一阶段：构建基于 Spring Boot 3 的多租户底层框架，确立 JWT 安全协议与 MyBatis Plus 数据隔离逻辑。

第二阶段：开发商品、销售、库存的基础业务模块，实现 CRUD 功能。

第三阶段：引入 FastAPI 与 LightGBM 算法，完成 AI 预测服务的跨语言集成。

第四阶段：利用 Vue 3 + Element Plus 构建前端界面，并集成 ECharts 实现数据可视化。

6.2 困难克服与技术突破

在项目实施过程中，我遇到了两个主要挑战：

异构系统的通信与同步：Java 后端与 Python AI 服务在数据结构和通信效率上存在差异。通过采用 RESTful 异步调用机制，并利用 JSON 序列化标准，成功实现了低延迟的数据交换。

大数据量下的性能瓶颈：初期的万级 CSV 数据导入常导致系统假死。通过引入 Spring Async 异步任务池与数据库批处理技术，将耗时任务从主线程剥离，并实时向前端推送任务状态，极大提升了系统的健壮性。

6.3 个人收获与能力提升

通过本次项目开发，我在以下方面取得了显著进步：

全栈架构思维：深刻理解了 SaaS 多租户架构的设计要点，掌握了从前端响应式

UI 到后端高并发处理的闭环开发能力。

AI 落地实践：不再仅停留在算法模型阶段，而是学会了如何将机器学习模型（LightGBM）封装为生产级服务，并与传统业务逻辑深度集成。

规范化开发：在技术文档编写、代码规范遵循及 RESTful API 设计方面形成了更专业的习惯。

6.4 后续升级与商业演进

本平台具备良好的扩展性，未来计划从以下维度进行优化：

算法模型升级：引入更先进的时序预测模型（如 LSTM 或 Transformer），进一步提高针对长周期销售波动的预测精度。

实时预警强化：集成 WebSocket 或第三方推送服务，实现库存预警、导入完成等信息的实时通知。

移动化适配：开发配套的微信小程序或移动 App，满足管理者随时随地查看经营大盘的需求。

商业价值推广：

中小企业赋能：本平台部署成本低、上手快，非常适合正处于数字化转型初期的小微零售商。

行业定制化：针对服装、生鲜等不同行业的库存周转特点，提供定制化的 AI 特征工程模板，实现垂直行业的 SaaS 服务渗透。

参考文献

【请按照标准参考文件格式填写】